

UNIVERSIDAD CATÓLICA DE CUENCA



**FACULTAD DE INFORMÁTICA, CIENCIAS DE LA COMPUTACIÓN E
INNOVACIÓN TECNOLÓGICA**

CARRERA DE ROBÓTICA E INTELIGENCIA ARTIFICIAL

ASIGNATURA: MICROCONTROLADORES

**TEMA: SISTEMA INTELIGENTE DE CONTROL DUAL CON ESP32
E INTERRUPTORES**

NOMBRE DEL ESTUDIANTE:

Mateo Roca, Jessica Ávila

DOCENTE: ING. PABLO BUESTÁN ANDRADE, PhD

FECHA: 7/05/2026

INTRODUCCIÓN

Los sistemas embebidos modernos requieren mecanismos eficientes para responder a eventos externos y señales analógicas en tiempo real. El uso de interrupciones y temporizadores de hardware permite construir aplicaciones reactivas sin depender de retardos bloqueantes. En esta práctica se implementa un sistema inteligente basado en el ESP32 capaz de gestionar entradas analógicas, eventos digitales y control de actuadores mediante una máquina de estados adaptativa.

OBJETIVO GENERAL

Implementar un sistema embebido inteligente utilizando el ESP32 que integre lectura analógica, control PWM, interrupciones externas y temporizadores de hardware para el control dinámico de actuadores.

OBJETIVOS ESPECIFICOS

- Implementar lectura analógica mediante potenciómetros usando `analogRead()`.
- Controlar actuadores utilizando `analogWrite()` y `digitalWrite()`.
- Configurar interrupciones externas mediante pulsadores.
- Implementar temporizadores de hardware para tareas periódicas.
- Utilizar comunicación serial para monitoreo del sistema en tiempo real.

DESARROLLO

Arquitectura de sistema

El sistema se ha diseñado bajo un paradigma de **programación dirigida por eventos**, eliminando por completo el uso de funciones bloqueantes como `delay()`. La arquitectura se basa en una **Máquina de Estados Reactiva** que gestiona tres estados principales: Desactivado, Modo 1 (PWM) y Modo 2 (Temporización por Hardware).

Configuración de Periféricos e Interrupciones

Para garantizar la respuesta en tiempo real, se configuraron los siguientes recursos del ESP32:

- **Interrupciones Externas (GPIO):** Se configuraron los pines 32 y 33 con resistencias *pull-up* internas. Estas interrupciones detectan el flanco de bajada (*FALLING*) de los pulsadores, permitiendo el cambio de modo y el encendido/apagado instantáneo del sistema mediante las funciones ISR `isrBtn1` y `isrBtn2`.
- **Temporizador de Hardware (Hardware Timer):** Se utilizó el periférico `hw_timer_t` del ESP32 configurado a una frecuencia base de **1 MHz**. Este timer genera una interrupción precisa que activa una bandera booleana (`timerFlag`), permitiendo que el parpadeo del LED 2 sea independiente de la carga de procesamiento del bucle principal.
- **Control de Potencia (PWM):** Se implementó la librería `analogWrite` (compatible con el core v3) para modular el ciclo de trabajo del LED 1 en función de la lectura del ADC del Potenciómetro 1.

Lógica de modos operativos

Modo 1: Control Inteligente PWM

En este modo, el sistema realiza un muestreo constante del **Potenci33metro 1 (Pin 34)** mediante la funci33n analogRead(). El valor de 12 bits (0-4095) se traduce mediante una funci33n de transferencia lineal (map) a un rango de 8 bits (0-255). Este valor resultante se aplica directamente al LED 1, permitiendo un control de intensidad fluido y visualmente lineal.

Modo 2: Sistema de Alerta Adaptativa

Este modo utiliza la **Base de Tiempo de Hardware**. El sistema lee el **Potenci33metro 2 (Pin 35)** y ajusta din33micamente la frecuencia de activaci33n de la alarma del timer (entre 1 Hz y 20 Hz). Cada vez que el timer alcanza el tiempo establecido, se ejecuta la ISR que conmuta el estado del LED 2. Se implement33 una l33gica de **hist33resis** para evitar que el ruido el33ctrico del potenci33metro provoque reconfiguraciones innecesarias del hardware.

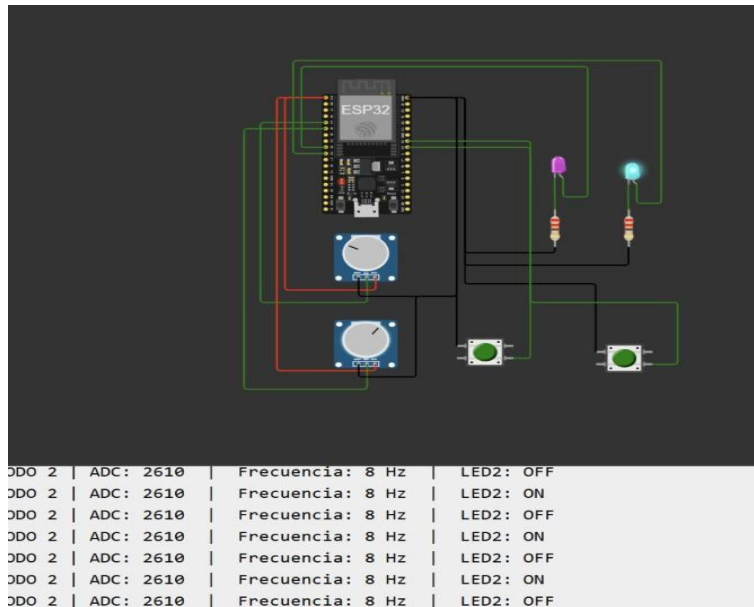
Gesti33n de Energ33a y seguridad

El sistema incluye un estado de "Desactivaci33n Cr33tica". Al presionar el pulsador 2, el sistema entra en una rutina de espera no bloqueante que apaga todos los actuadores y suspende el procesamiento de los modos 1 y 2, manteniendo 33nicamente activas las interrupciones de control. Esto asegura un consumo m33nimo y la seguridad de los componentes.

Asignaci33n de pines

Componente	Pin ESP32	Funci33n
Potenci33metro 1	GPIO 34	Entrada Anal33gica (Modo 1)
Potenci33metro 2	GPIO 35	Entrada Anal33gica (Modo 2)
LED 1	GPIO 25	Salida PWM (Intensidad)
LED 2	GPIO 26	Salida Digital (Parpadeo)
Pulsador 1	GPIO 32	Interrupci33n (Cambio Modo)
Pulsador 2	GPIO 33	Interrupci33n (Sistema ON/OFF)

DISEÑO ESQUEMATICO



CODIGO

```
const int pot1Pin = 34;
const int pot2Pin = 35;
const int led1Pin = 25;
const int led2Pin = 26;
const int btnModo = 32;
const int btnPwr = 33;

#define LEDC_FREQ_HZ 5000
#define LEDC_BITS 8

volatile bool timerFlag = false;
volatile bool estadoSistema = true;
volatile int modoActual = 1;
volatile unsigned long lastBtnModo = 0;
volatile unsigned long lastBtnPwr = 0;
#define DEBOUNCE_US 25000UL
volatile bool estadoLed2 = false;

hw_timer_t* timer = NULL;
```

```

void IRAM_ATTR onTimer() {
    timerFlag = true;
}

void IRAM_ATTR ISR_CambiarModo() {
    unsigned long ahora = micros();
    if (ahora - lastBtnModo > DEBOUNCE_US) {
        lastBtnModo = ahora;
        if (estadoSistema) {
            modoActual = (modoActual == 1) ? 2 : 1;
        }
    }
}

void IRAM_ATTR ISR_ToggleSistema() {
    unsigned long ahora = micros();
    if (ahora - lastBtnPwr > DEBOUNCE_US) {
        lastBtnPwr = ahora;
        estadoSistema = !estadoSistema;
    }
}

void configurarTimer(uint32_t freqHz) {
    if (freqHz < 1) freqHz = 1;

    if (timer != NULL) {
        timerStop(timer);
    } else {
        timer = timerBegin(1000000);
        timerAttachInterrupt(timer, &onTimer);
    }

    uint64_t ticks = 1000000UL / (freqHz * 2);
    timerAlarm(timer, ticks, true, 0);
    timerWrite(timer, 0);
    timerStart(timer);
}

void setup() {
    Serial.begin(115200);

    pinMode(led1Pin, OUTPUT);
    pinMode(led2Pin, OUTPUT);
    pinMode(btnModo, INPUT_PULLUP);
    pinMode(btnPwr, INPUT_PULLUP);
}

```

```

    ledcAttach(led1Pin, LEDC_FREQ_HZ, LEDC_BITS);

    attachInterrupt(digitalPinToInterrupt(btnModo), ISR_CambiarModo, FALLING);
    attachInterrupt(digitalPinToInterrupt(btnPwr), ISR_ToggleSistema, FALLING);

    configurarTimer(2);
    Serial.println("=== SISTEMA CORREGIDO Y LISTO ===");
}

void loop() {

    if (!estadoSistema) {
        ledcWrite(led1Pin, 0);
        digitalWrite(led2Pin, LOW);

        static unsigned long lastOffMsg = 0;
        if (millis() - lastOffMsg > 1000) {
            Serial.println("SISTEMA DESACTIVADO");
            lastOffMsg = millis();
        }
        return;
    }

    if (modoActual == 1) {
        digitalWrite(led2Pin, LOW);
        int valorADC = analogRead(pot1Pin);
        int valorPWM = map(valorADC, 0, 4095, 0, 255);
        ledcWrite(led1Pin, valorPWM);

        static unsigned long lastLog1 = 0;
        if (millis() - lastLog1 > 300) {
            Serial.printf("MOD0 1 | ADC: %d | PWM: %d\n", valorADC, valorPWM);
            lastLog1 = millis();
        }
    }

    else if (modoActual == 2) {
        ledcWrite(led1Pin, 0);

        int valorPot2 = analogRead(pot2Pin);
        int freqParpadeo = map(valorPot2, 0, 4095, 20, 1);

        static int freqAnterior = 0;

```

```

if (abs(freqParpadeo - freqAnterior) >= 1) {
    configurarTimer(freqParpadeo);
    freqAnterior = freqParpadeo;
}

if (timerFlag) {
    timerFlag = false;
    estadoLed2 = !estadoLed2;
    digitalWrite(led2Pin, estadoLed2);

    Serial.printf("MOD0 2 | Freq: %d Hz | LED: %s\n", freqParpadeo, estadoLed2
? "ON" : "OFF");
}
}
}
}

```

CONCLUSIONES

- Se logró implementar una célula robótica autónoma donde el control del tiempo no depende del software, sino del hardware interno del ESP32, asegurando que los procesos de manufactura duren exactamente lo programado sin retrasos por otros cálculos del programa.
- El uso de interrupciones externas demostró ser fundamental para la seguridad y eficiencia industrial, ya que el sistema es capaz de detectar la llegada de material al instante, sin necesidad de estar preguntando constantemente el estado del botón en el código principal.
- La estructura de máquina de estados permitió un flujo de trabajo organizado y seguro, impidiendo que el robot intente inspeccionar una pieza que aún no ha sido ensamblada, garantizando así la integridad del proceso de fabricación.

FOTOGRAFIAS

